

A2Q2

Lily Li

2025-06-09

Q2

(a)

```
make_bezier_path_1free = function(x=c(3,5), point2=c(7,5), begin=c(0,0), end=c(10,2))
{
  px = c(begin[1], x[1], point2[1], end[1])
  py = c(begin[2], x[2], point2[2], end[2])

  t = seq(from=0, to=1, by=0.01)
  bezier_path_x = (1 - t)^3 * px[1] +
    3 * (1 - t)^2 * t * px[2] +
    3 * (1 - t) * t^2 * px[3] +
    t^3 * px[4]

  bezier_path_y = (1 - t)^3 * py[1] +
    3 * (1 - t)^2 * t * py[2] +
    3 * (1 - t) * t^2 * py[3] +
    t^3 * py[4]

  return(list(x=bezier_path_x, y=bezier_path_y))
}

get_path_badness = function(path, factor_acc=10, factor_dist=1, factor_cdir=20) {
  x = path$x
  y = path$y
  dx = diff(x)
  dy = diff(y)
  d2x = c(diff(dx), 0)
  d2y = c(diff(dy), 0)
  acc = sqrt(d2x^2 + d2y^2)
  ds = sqrt(dx^2 + dy^2)
  dir = atan2(dy, dx)
  ddir = abs(diff(dir))
  badness = factor_acc * max(acc)^2 + factor_dist * sum(ds)^2 + factor_cdir * max(ddir)^2
  return(badness)
}

# Makes the data frame to use
x = rep(seq(0,10,0.1), times=101)
```

```

y = rep(seq(0,10,0.1), each=101)
df_1a = data.frame(x,y)
df_1a$rho = NA

for (k in 1:nrow(df_1a)) {
  guide1 = c(df_1a$x[k], df_1a$y[k])
  path = make_bezier_path_1free(x = guide1)
  df_1a$rho[k] = get_path_badness(path)
}

head(df_1a)

```

```

##      x y      rho
## 1 0.0 0 119.2592
## 2 0.1 0 119.6741
## 3 0.2 0 119.6252
## 4 0.3 0 119.4553
## 5 0.4 0 119.2971
## 6 0.5 0 119.1674

```

```
tail(df_1a)
```

```

##      x y      rho
## 10196 9.5 10 208.0083
## 10197 9.6 10 208.1604
## 10198 9.7 10 208.3239
## 10199 9.8 10 208.4991
## 10200 9.9 10 208.6858
## 10201 10.0 10 208.8844

```

(b)

```

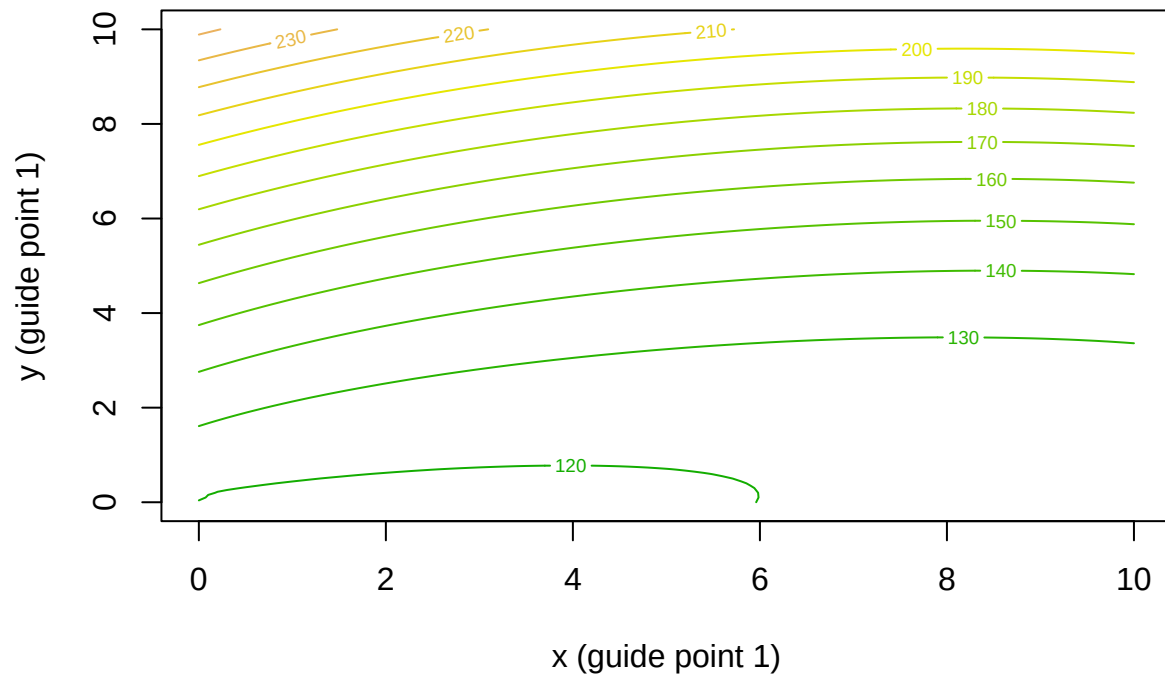
x_vals <- unique(df_1a$x)
y_vals <- unique(df_1a$y)

z_matrix <- matrix(df_1a$rho, nrow=101, ncol=101, byrow=FALSE)

contour(x_vals, y_vals, z_matrix,
        xlab = "x (guide point 1)",
        ylab = "y (guide point 1)",
        main = "Contour Plot of Path Badness",
        col = terrain.colors(20))

```

Contour Plot of Path Badness



(c)

```
x <- df_1a$x
y <- df_1a$y
rho <- df_1a$rho

X <- cbind(1, x, x^2, x^3, y, y^2, y^3, x*y, x^2 * y, x * y^2)
Y <- rho

XtX <- t(X) %*% X
XtY <- t(X) %*% Y
beta_hat <- solve(XtX) %*% XtY
beta_hat
```

```
##          [,1]
## 119.62476202
## x  -1.41712836
##    0.33197656
##   -0.01425001
## y   4.84700303
##    0.83065480
##   -0.00850790
##   -1.14061235
```

```
##      0.04450075
##      0.03095690
```

```
model_check <- lm(rho ~ x + I(x^2) + I(x^3) + y + I(y^2) + I(y^3) + I(x*y) + I(x^2*y) + I(x*y^2), data =
summary(model_check)$coef
```

```
##              Estimate   Std. Error   t value Pr(>|t|)
## (Intercept) 119.62476202 1.629379e-02 7341.73728      0
## x           -1.41712836 8.388001e-03 -168.94709      0
## I(x^2)       0.33197656 1.643687e-03  201.97064      0
## I(x^3)      -0.01425001 1.037853e-04 -137.30277      0
## y            4.84700303 8.388001e-03  577.84958      0
## I(y^2)       0.83065480 1.643687e-03  505.36063      0
## I(y^3)      -0.00850790 1.037853e-04  -81.97596      0
## I(x * y)     -1.14061235 1.310348e-03 -870.46551      0
## I(x^2 * y)   0.04450075 9.111992e-05  488.37560      0
## I(x * y^2)   0.03095690 9.111992e-05  339.73802      0
```

(d)

```
rho_fn = function(x=c(x0, y0)) {

  path <- make_bezier_path_1free(x = x)
  rho <- get_path_badness(path)

  return(rho)
}
```

(e)

```
opt_result <- optim(par = c(5, 5), fn = rho_fn, method = "L-BFGS-B", lower = c(0, 0), upper = c(10, 10))
opt_result$par # best x, y
```

```
## [1] 2.024801 0.000000
```

```
opt_result$value # minimum rho
```

```
## [1] 118.6237
```

(f)

```

make_bezier_path_2free = function(x, begin=c(0,0), end=c(10,2))
{
  point1 <- c(x[1], x[2])
  point2 <- c(x[3], x[4])

  px = c(begin[1], point1[1], point2[1], end[1])
  py = c(begin[2], point1[2], point2[2], end[2])

  t = seq(from=0, to=1, by=0.01)
  bezier_path_x = (1 - t)^3 * px[1] +
    3 * (1 - t)^2 * t * px[2] +
    3 * (1 - t) * t^2 * px[3] +
    t^3 * px[4]

  bezier_path_y = (1 - t)^3 * py[1] +
    3 * (1 - t)^2 * t * py[2] +
    3 * (1 - t) * t^2 * py[3] +
    t^3 * py[4]

  return(list(x=bezier_path_x, y=bezier_path_y))
}

```

(g)

```

rho_fn2 = function(x){
  path <- make_bezier_path_2free(x)
  rho <- get_path_badness(path)
  return(rho)
}

start <- c(3, 5, 7, 5)

opt_result_2free <- optim(par = start, fn = rho_fn2, method = "L-BFGS-B", lower = c(0, 0, 0, 0), upper = c(10, 10, 10, 10))

opt_result_2free$par      # Best positions for (x1, y1, x2, y2)

## [1] 5.700401 1.140086 6.243891 1.248783

opt_result_2free$value    # Minimum badness

## [1] 104.0001

```